

Mysterious Parody Bits

Lab 1 - COMP211 - Fall 2026

"How often have I said to you that when you have eliminated the impossible, whatever remains, however improbable, must be the truth?"

~ Sherlock Holmes, *The Sign of Four* (1890)

Part 2 of 3. Doughnut Holes and parity Bits Overview

Perhaps you've noticed US ASCII is a 7-bit encoding although a `char` can hold 8-bits. The 8th bit is often used for locale-specific encodings, such as fancy Umlaut marks. Since the mystery you are solving contains highly sensitive messages, you'll instead use the 8th bit for *parity*, a technique to help detect corrupted data.

A Parity Parody

Ancient lore has it if you squint and look closely at your computer's memory you just might find each `char` is made of tiny doughnuts and french toast sticks, often represented as 0s and 1s. If you do find this, you should eat them, that'd be one delicious byte.

The idea of *even parity* is you use a spare bit to always have an even number of french toast sticks in a bitfield. After encoding with even parity, if you later count an odd number of french toast sticks in a byte, then you've detected data *corruption*.

Computing the Parity Bit

Computers store data using bits (0 and 1). In standard ASCII, characters use 7 bits, even though a `char` has 8 bits. The extra bit can be used as a parity bit, which helps detect errors. This is a real technique that is been used in memory systems and communication protocols.

With even parity, the total number of 1 bits in the byte must be even.

To compute even parity:

- Look at the 7-bit US-ASCII value of the character
- Count how many 1 bits it contains
- If the count is odd, the parity bit is 1
- If the count is even, the parity bit is 0

After adding the parity bit, the total number of 1s will always be even.

Where We Store the Parity Bit

In this lab, we use a non-standard parity layout. This does not provide real security, it just makes the exercise more interesting. Instead of placing the parity bit in the most significant bit, we place it in the least significant bit.

To do this:

- Compute the parity bit as described above
- Shift the 7-bit US-ASCII value left by one bit
- Store the parity bit in the lowest-order bit
- The final 8-bit value looks like this:

[ASCII bit 6][ASCII bit 5][ASCII bit 4][ASCII bit 3][ASCII bit 2][ASCII bit 1][ASCII bit 0][parity bit]

Character	ASCII (hex)	7-bit Binary	# of 1s	Parity Bit	8-bit w/ Parity	Hex
1	31	0110001	3 (odd)	1	01100011	63
3	33	0110011	4 (even)	0	01100110	66
C	43	1000011	3 (odd)	1	10000111	87
Z	5A	1011010	4 (even)	0	10110100	B4

Let's Begin: implementing is_odd

Complete your work for this part of the lab in `parity.c`.

We'll start with first determining if the number of bits in the input character is odd.

Open `parity.c` and implement the `is_odd` function. This function should: - Return 1 if the input's binary representation contains an odd number of 1s. - Return 0 if it contains an even number of 1s.

For example, the character `C` has the ASCII value `0x43`, which is `0b1000011` in binary. This binary value contains three 1s. Since 3 is odd, `is_odd` should return `1`.

You must use bitwise operators to implement this function.

Writing Tests for is_odd

In this section, you will write simple tests to check whether your `is_odd` function correctly determines if the number of `1` bits in a character is odd.

Each test consists of:

- an input file (`.in`)
- an expected output file (`.out`)

You will run your program using the input file and compare the output against the expected output using `diff`.

Step 1: Create a directory for the tests

```
learncli$ mkdir -p tests/parity/is_odd/{input,expected,actual}
```

Step 2: Create the input file.

This test uses only digits, which are safe printable characters.

The input file contains `123` with no newline at the end. Since `echo` normally adds a newline, use the `-n` option to prevent it from adding the newline:

```
echo -n "123" > tests/parity/is_odd/input/digits.in
```

Step 3: Create the expected output file.

Based on the ASCII values of the digits:

- '1' has an odd number of 1 bits: output 1
- '2' has an odd number of 1 bits: output 1
- '3' has an even number of 1 bits: output 0

```
learncli$ printf "1: \n2: \n3: 0\n" > tests/parity/is_odd/expected/digits.expected
```

Step 4: Compile your program

```
learncli$ gcc parity.c -Wall -Wextra -std=c11 -g -o is_odd_prog
```

Make sure there are no compiler errors.

Step 5: Run the test manually

Run your program with input redirected from the test file and save the output:

```
learncli$ ./is_odd_prog < tests/parity/is_odd/input/digits.in > tests/parity/is_odd/actual/digits.out
```

Compare the output to the expected output using `diff`.

Completing parity.c

Now that `is_odd` is working, you can use it as a building block for the rest of the parity encoder.

Your task is to:

- Read input one character at a time
- Determine whether the 7-bit US-ASCII value has an odd or even number of 1 bits
- Compute the correct parity bit
- Construct the final 8-bit value by shifting and inserting the parity bit
- Output the resulting encoded character
- Output a newline character after processing all input

Testing parity.c

In this part, your program reads normal ASCII text and outputs parity-encoded bytes. Many of those bytes will not print as readable characters, so we will test parity in two simple ways:

- Digits-only tests (these produce readable output)
- Use your hex program to turn the output into readable hex

You will create test files and compare your output to expected output using `diff`, just like before.

Step 1: Compile parity

From your project directory:

```
gcc parity.c -Wall -Wextra -std=c11 -g -o parity
```

Make sure there are no compiler errors.

Step 2: Making your first test.

Create the test directories

```
learncli$ mkdir -p tests/parity/{input,expected,actual}
```

Test A: Digits-only (readable output)

When the input contains only digits, the output of `parity` is still printable characters. This makes a great first test.

Create the input file

```
learncli$ echo -n "123456789" > tests/parity/input/digits.in
```

Create the expected output file

```
learncli$ printf "cefijloqr" > tests/parity/expected/digits.out
```

Note: no newline at the end of the expected output file.

Run the test

```
learncli$ ./parity < tests/parity/input/digits.in > tests/parity/actual/digits.out
learncli$ diff tests/parity/actual/digits.out tests/parity/expected/digits.out
```

Test B: General text (test using `hex`)

For letters and punctuation, the parity-encoded bytes may not be readable. So we will test by piping the output of `parity` into your hex program.

Create the input file

```
echo -n "abcdef" > tests/parity/input/abcdef.in
```

Create the expected output file

From the lab example, the correct result is:

```
echo "C3C5C6C9CACC" > tests/parity/expected/abcdef.out
```

Run the test

```
./parity < tests/parity/input/abcdef.in | ./hex > tests/parity/actual/abcdef.out
diff tests/parity/actual/abcdef.out tests/parity/expected/abcdef.out
```

Makefiles make our lives easier

Up to this point, you have been compiling your programs by typing long `gcc` commands by hand. That works, but it is easy to make mistakes and it becomes annoying as the number of files grows.

A `Makefile` lets you:

- save compilation commands in one place
- rebuild programs with a single command
- avoid recompiling files that have not changed

This is a standard tool used in real software projects.

Please read [Dive into Systems Section 17.15.1 to 17.15.6](#). Then, answer the corresponding questions on [Gradescope](#).

Now let's create a Makefile for this lab!

Step 1: From the root of your project directory, create a file named `Makefile` (capital M):

```
touch Makefile
```

Open it in your editor:

```
code Makefile
```

Step 2: Add basic build rules

Paste the following into your `Makefile`:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -g

all: xeh hex

xeh: xeh.c
    $(CC) $(CFLAGS) -o xeh xeh.c

hex: hex.c
    $(CC) $(CFLAGS) -o hex hex.c

clean:
    rm -f xeh hex
```

Let's break this down:

Variable definitions

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -g
```

- These lines define variables that can be reused later in the `Makefile`.
- `CC` stores the name of the C compiler
- `CFLAGS` stores compiler options

The all target

```
all: xeh hex
```

This is a target named `all`.

When you run:

```
learncli$ make
```

`make` builds the first target, which is `all`.

So this line tells make to compile both programs.

The xeh target

```
xeh: xeh.c
    $(CC) $(CFLAGS) -o xeh xeh.c
```

This rule explains how to build the `xeh` program.

- Target: `xeh`
- Dependency: `xeh.c`

Meaning:

If `xeh.c` changes (or `xeh` doesn't exist), rebuild `xeh`.

The command:

```
$(CC) $(CFLAGS) -o xeh xeh.c
```

Expands to:

```
gcc -Wall -Wextra -std=c11 -g -o xeh xeh.c
```

Try running make

First, delete your executable files

```
learncli$ rm xeh hex parity
```

Then, from the root of your repo, run:

```
learncli$ make
```

You should see `gcc ...` commands run and end up with two executables in your directory:

```
learncli$ ls
Makefile  xeh.c  hex.c  xeh  hex  ...
```

Now try running `make` again:

```
learncli$ make
```

This time you should see something like:

```
make: Nothing to be done for 'all'.
```

That's because `make` checks timestamps and does not rebuild targets that are already up to date.

To see this more clearly, modify one `xeh.c` (even just add a comment), then rebuild:

```
learncli$ make xeh
```

Only `xeh` should rebuild.

Finally, remove compiled files using the `clean` rule:

```
learncli$ make clean
```

Then confirm they are gone:

```
learncli$ ls
Makefile  xeh.c  hex.c  ...
```

Adding a Parity Target

First, update the `Makefile`'s all target to include `parity`:

```
all: xeh hex parity
```

Then add a new rule for `parity`.

```
parity: parity.c
    $(CC) $(CFLAGS) -o parity parity.c
```

Also update `clean` so it removes the `parity` executable too:

```
clean:
    rm -f xeh hex parity
```

To a great mind, nothing is little

Once your `parity` encoding is working, pipe the output of echoing the following string into your parity encoder and then pipe its output into your hex encoder. There **should** be a newline at the end of this string, so **don't** use the `-n` option with `echo`.

nothing is little

Take the resulting secret value and navigate to <http://bit.ly/your-encoding-here> to reach the final part of this lab.